



Синтез и анализ на алгоритми за факултети ФКСТ, ФТК, ФЕТТ

проф. д-р Даниела Минковска
каб. 2324, кат. ПКТ, ФКСТ, ТУ-София
daniela@tu-sofia.bg



Лекция 4

Структури от данни. Графи – видове, представяне, обхождане, алгоритми за обхождане в дълбочина, ширина, определяне на минимално скелетно дърво.

“АЛГОРИТМИ + СТРУКТУРИ ОТ ДАННИ = ПРОГРАМИ”

Н. УИРТ

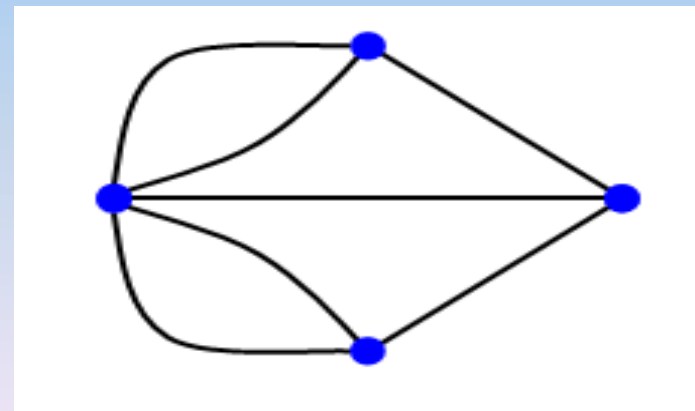
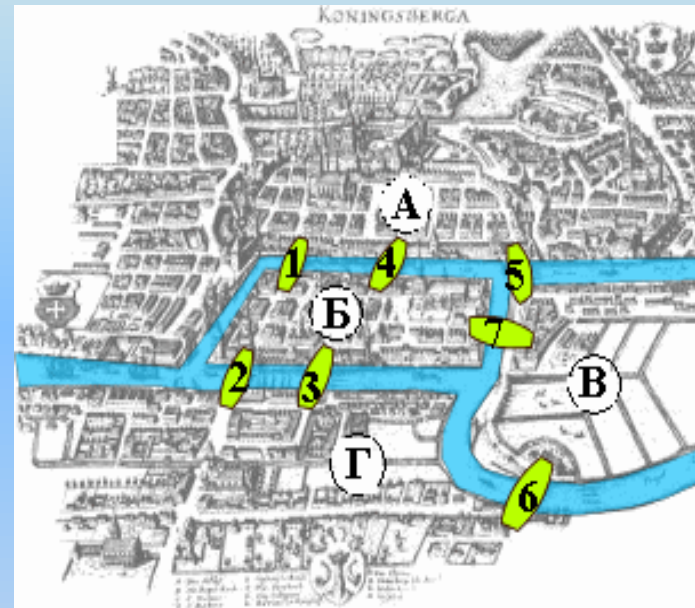
Графи. Исторически преглед

Исторически първият проблем, който може да бъде отнесен към теория на графите, е задачата за **Кьонигсбергските мостове на Ойлер** (1736).

През пруският град Кьонинсберг минава реката Прегел, на която са построени **седем моста**. Пет от тях свързвали намиращия се в града остров с двата бряга.

Въпросът е: дали е възможно да бъде извършена такава разходка през града, че всеки мост да бъде преминат точно веднъж?

Отговаряйки на този въпрос, Ойлер поставя основите на теория на графите.





Графи. Основни понятия

Граф - дефинира се чрез съвкупност от **върхове** (нар. още **възли**) и съвкупност от **вързки** между отделни двойки върхове.

Връзките могат да имат **посока**, или да нямат – **ненасочени**. Връзките се наричат **дъги** (**клони** или **ребра**).

Връзките обикновено се асоциират със свързващи пътища (далекопроводи и пр.) между върховете, а чрез самите върхове в приложенията се моделират градове, гари, водохранилища, електрически подстанции и т.н. В приложенията към всяка дъга или ребро се приписват една или повече **числови стойности** (нар. още **тегла**), които имат смисъл да зададат дължината на разстоянието между двата свързани върха, цената за преминаване, максимален пропускователен капацитет и пр.

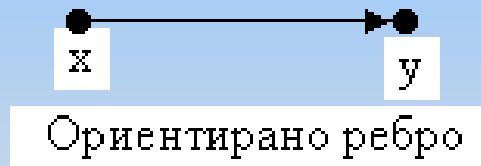


Графи. Определения

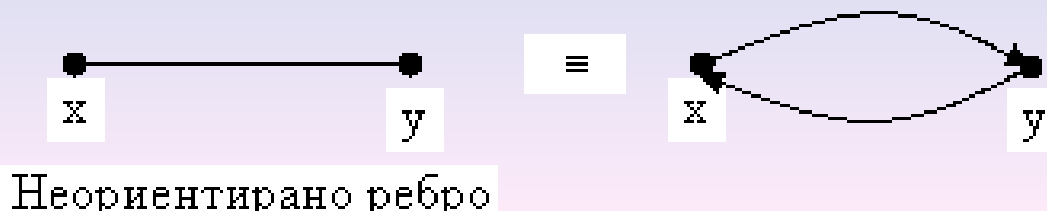
Граф - **нелинейна**, динамична структура от данни, изградена от **възли** (върхове) и връзки между тях наречени **ребра** (клони, дъги).

Нагледно графът се изобразява с множество от точки, свързани с линии. С (x,y) се означава клон и се казва, че възлите x и y са съседни възли

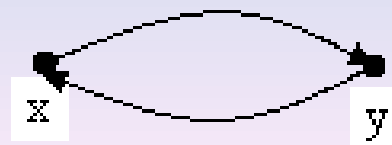
Когато двойката е **наредена**, т.е. $(x,y) \neq (y,x)$, то реброто се нарича **ориентирано**.



Когато двойката е **ненаредена**, т.е. $(x,y) = (y,x)$, то реброто се нарича **неориентирано**. Всяко неориентирано ребро може да се представи като съвкупност от две ориентирани ребра с различни посоки.



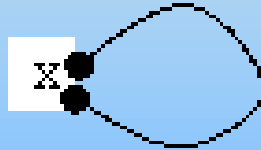
≡



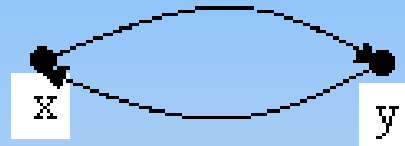


Графи. Определения

Бримка – ребро, за което началния и крайния възел съвпадат.



Паралелни ребра – ребра, които свързват едни и същи възли.



Обикновен граф (или само граф) се нар. граф, съдържащ само неориентирани клони без бримки и паралелни клони

Графи. Определения. Неориентиран граф

Дефиниция: Краен **неориентиран** граф се нарича **наредената двойка (V, E)** , където:

$V = \{v_1, v_2, \dots, v_n\}$ – крайно множество от върхове;

$E = \{e_1, e_2, \dots, e_m\}$ – крайно множество от неориентирани ребра, като всеки елемент $e_k \in E$ принадлежи на E ,

$(k = 1, 2, \dots, m)$ е ненаредена двойка (v_i, v_j) ,

където $v_i, v_j \in V$, $1 \leq i, j \leq n$.

Ако освен това е зададена функция $f(i, j)$, съпоставяща целочислена стойност на всяко ребро $(i, j) \in E$, $f(i, j) = f(j, i)$, графът се нарича **претеглен неориентиран граф**.



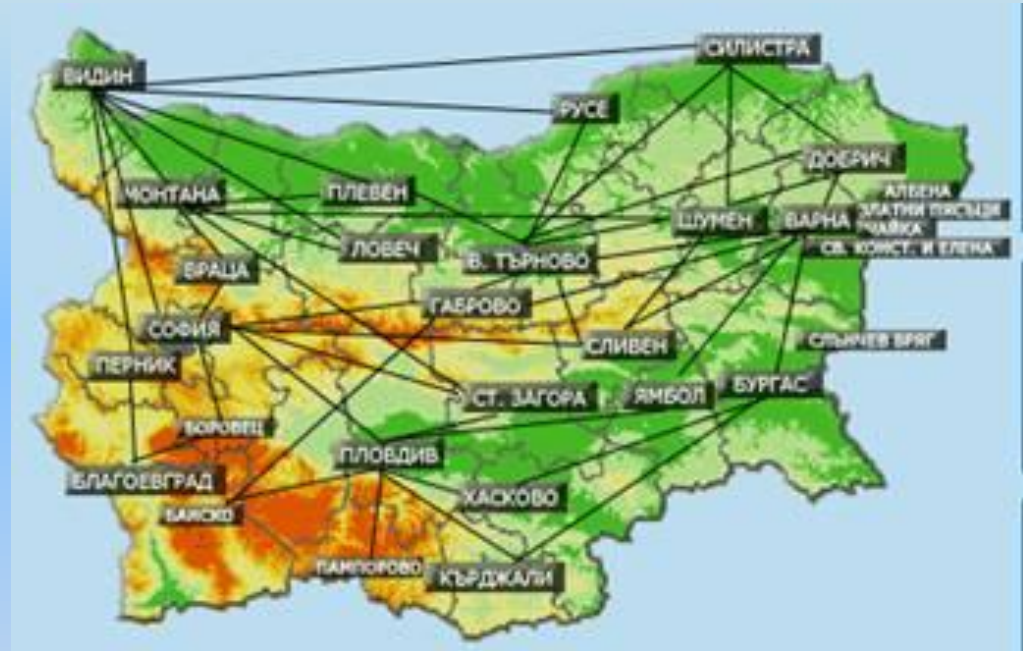
Графи. Определения. Неориентиран граф

Степен на възел на неориентиран граф – се нарича броят на клоните, свързани с възела (броят на съседните му възли)

- ***Изолиран възел – възел, със степен равна на нула;***
- ***Висящ възел – възел със степен равна на едно и клоната свързан с него.***

Графи. Неориентиран граф - примери

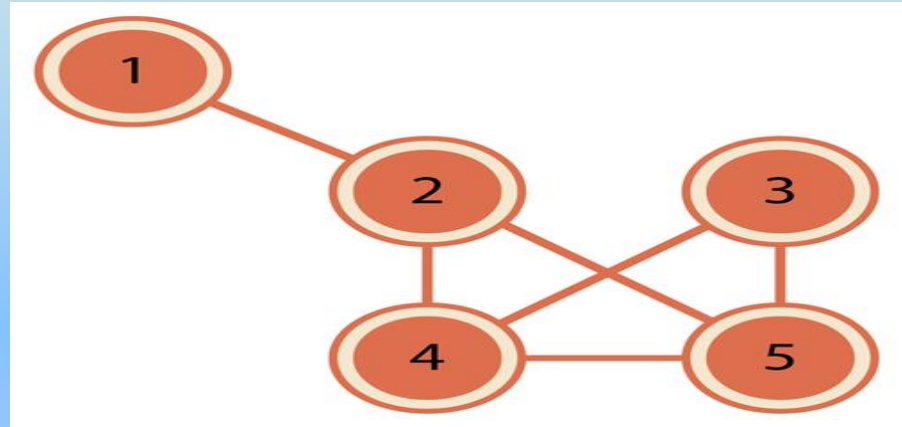
Пр1. *Пътна мрежа (транспортна карта) на район (България) – тегловен граф, с възли – градовете (или кръстовища), теглата са разстоянията.*



Пр2. *Компютърна мрежа – неориентиран граф, в който компютрите са възли (върхове), а всяко ребро между два върха показва, че компютрите са свързани в мрежа.*

Графи. Неориентиран граф - примери

Пр3. Множество от страници в Internet и връзките по между им.



Пр4. Химични връзки (напр. молекула) - атомите са възли, теглата им са видовете елементи, а клоните са връзките между атомите, като валентностите на тези връзки са теглата на клоните.

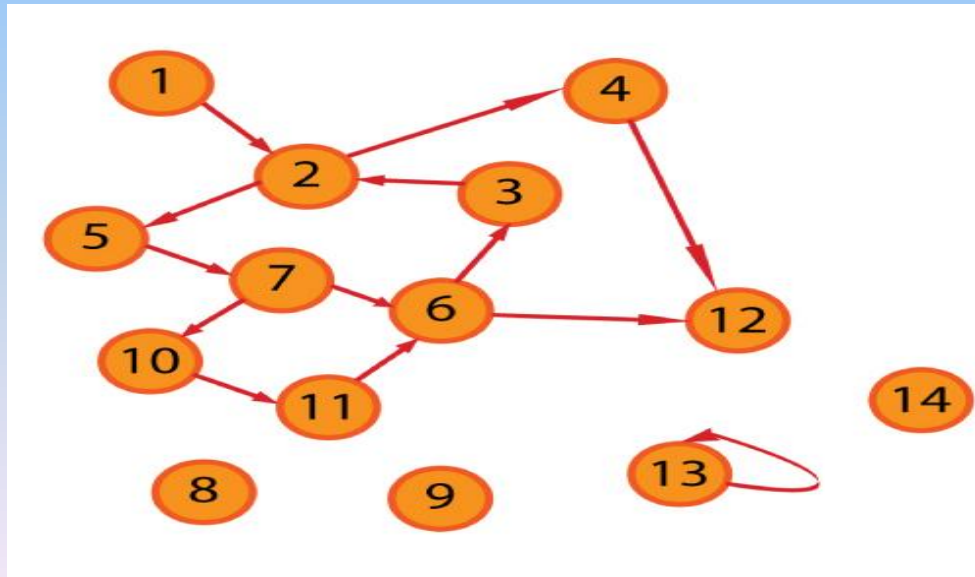
Графи. Определения. Ориентиран граф

Дефиниция: Краен **ориентиран** граф се дефинира като **съвкупност** от **две множества** – множеството на възлите **V** и множеството на ребрата **E** . т.е. **$G(V,E)$** , където:

$V = \{v_1, v_2, \dots, v_n\}$ – крайно множество от върхове;

$E = \{e_1, e_2, \dots, e_m\}$ – крайно множество от ориентирани ребра, като всеки елемент e_k принадлежи на E , ($k = 1, 2, \dots, m$) е наредена двойка **(v_i, v_j)** , където

$v_i, v_j \in V$, $1 \leq i, j \leq n$.

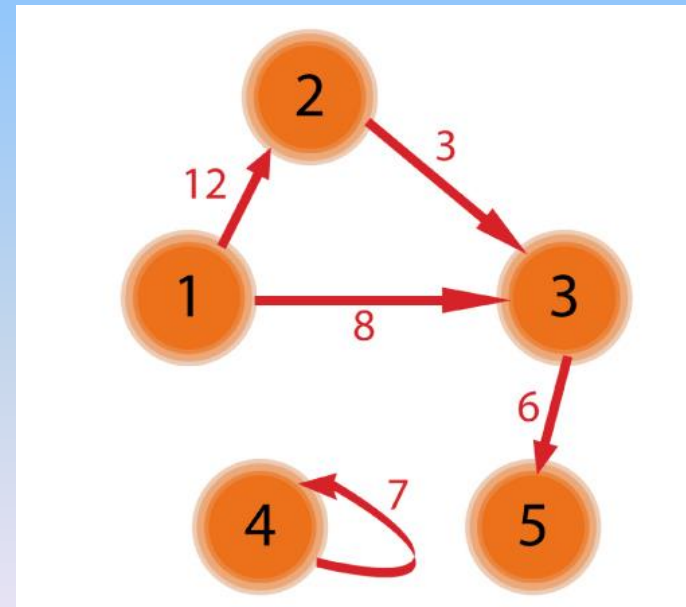


Графи. Определения. Ориентиран граф

Ако в допълнение е дадена числова функция f :

$E \rightarrow R$, съпоставяща на всяко ребро e_k **тегло** $f(e_k)$, графът се нарича **претеглен**. Най-често, ориентиран граф се представя графично в равнината чрез множество от точки (кръгчета), означаващи върховете му и свързващи ги стрелки - ребра на графа.

(Някои автори използват термина **дъга** за **ориентирано** ребро, а **ребро** - за **неориентирано** ребро.)





Графи. Определения

- **Мултиграф** - Даден е ориентиран (неориентиран) граф $G(V,E)$. Ако в множеството на ребрата му се допуска **повторение** (т.е. E е мултимножество), G се нарича мултиграф.
- Даден е ориентиран граф $G(V,E)$. Два върха i и j ($i, j \in V$) се наричат **съседни**, ако поне едно от ребрата (i,j) и (j,i) принадлежи на E . В такъв случай още казваме, че i и j са **краища** за ребрата (i,j) и (j,i) .

За всяко ребро (i,j) **върхът i се нарича предшественик на j** , а j - **наследник на i** . Всеки от върховете i и j се нарича **инцидентен** с реброто (i,j) . Казваме, че **две ребра са инцидентни, когато са инцидентни с един и същи връх**. В случай на неориентиран граф, понятията съседен връх, край на ребро, инцидентност на върхове и ребра се дефинират аналогично.

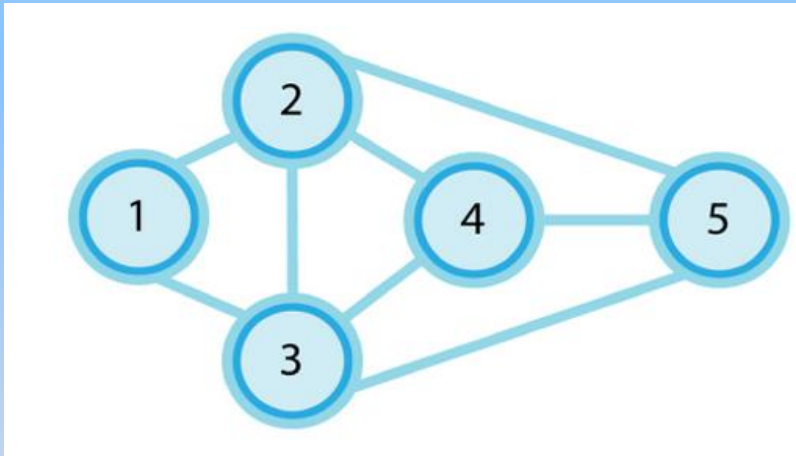


Графи. Определения

- **Път** в ориентиран (неориентиран) граф $G(V, E)$ се нарича **последователност от върхове** $v_1, v_2, \dots, v_k$ такива, че за всяко $i = 1, 2, \dots, k-1$ е **изпълнено** $(v_i, v_{i+1}) \in E$. Върховете v_1 и v_k се наричат **краища** на пътя. Ако $v_1 = v_k$, пътят се нарича **цикъл**. Ако за всяко $i \neq j (1 \leq i, j \leq k) \Rightarrow v_i \neq v_j$, то пътят се нарича **прост**. Съответно, ако $v_1 = v_k$, и всички останали върхове са различни, цикълът се нарича **прост**. Когато граф съдържа **поне един цикъл**, той се нарича **цикличен**, а в противен случай – **ацикличен**

Графи. Определения

- **Цикъл (контур)** - път, за който началния и крайния възел съвпадат.
- **Прост цикъл** – цикъл, в който няма повтарящи се възли.



Пътища:

П1: 5-4-2-1

П2: 1-2-3-4-5

П3: 1-2-4-3-2-1

П4: 1-2-4-3-1

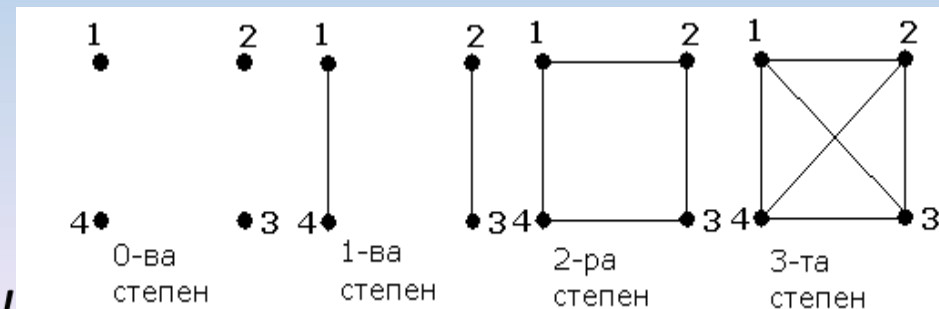
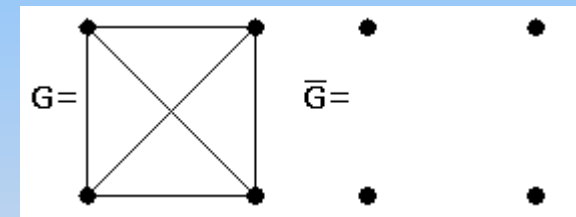
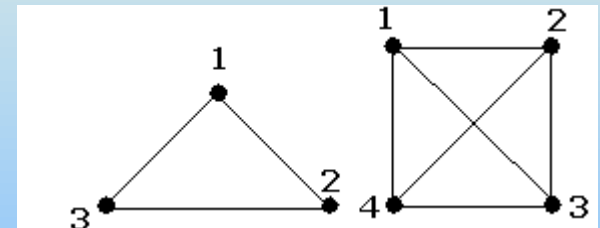
П3 и П4 - цикли

П4 - прост цикъл

- **Хамилтонов цикъл** – цикъл, който включва всички възли на графа точно по веднъж (преминава се през всеки възел точно по веднъж)
- **Ойлеров цикъл** – цикъл, който включва всички ребра на графа точно по веднъж. (преминава се през всеки клон по веднъж)

Графи. Видове

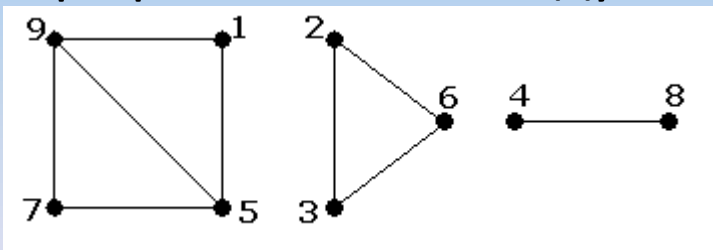
- **Пълен** - граф, в който има клон между всички двойки възли. Има $(n-1)/2$ клона, т.к. всеки възел има $(n-1)$ съседни и всеки клон се брои при двата възела които свързва.
- **Празен** – граф, който няма нито един клон.
- **Допълнителен граф $\bar{G}(V_1, E_1)$** на граф $G(V, E)$, се нар. граф, в който $V_1 = V$ и ако между два възела в G има клон, то в $\bar{G}$ няма и обратно.
- **Регулярен (еднороден)** – граф, на който всички възли имат еднаква степен.



Пълният граф е винаги регулярен!

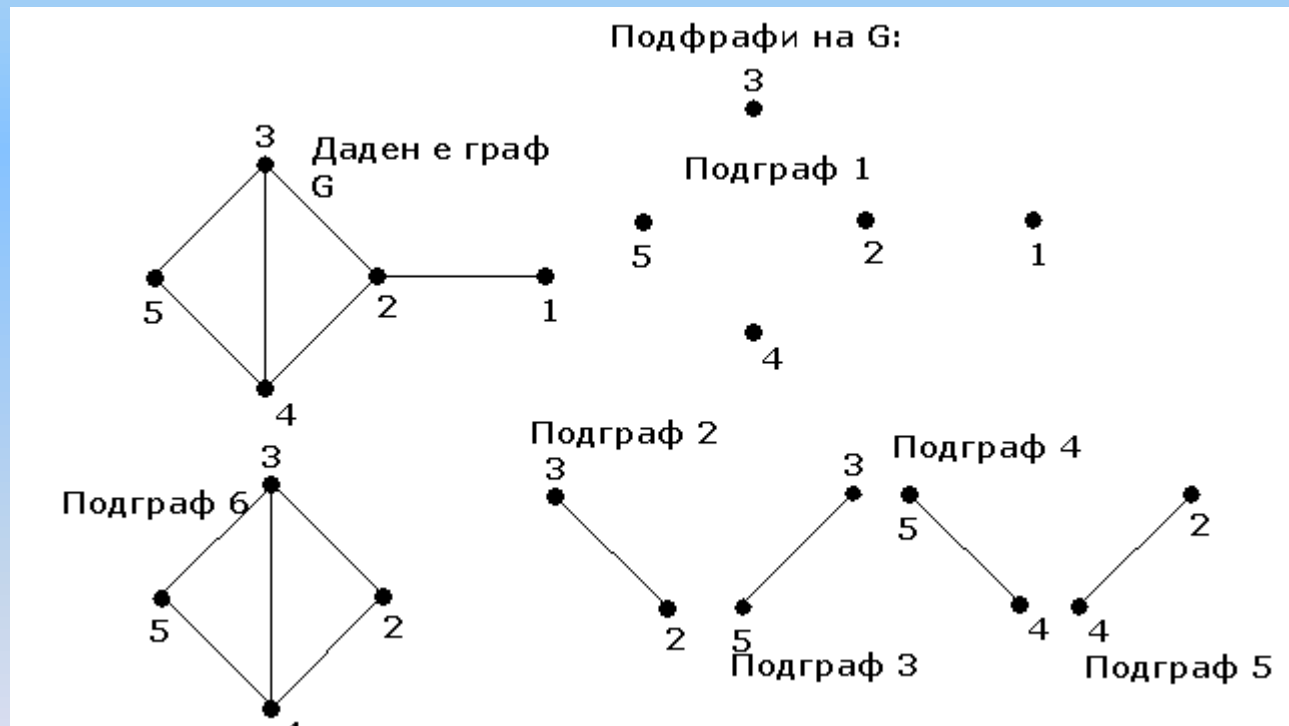
Графи. Видове

- **Тегловен** - граф, на чиито възли или/и клони има присвоени тегла.
- **Равнинен (планарен)** – граф, който се изобразява в равнината без пресичане на клони.
- **Двудялов** - граф, на който възлите са разделени на две подмножества V_1 и V_2 (двата дяла) и има клони само между възли от двата дяла.
 $G(V,E) \quad V = V_1 + V_2$
 $\text{За } \forall (x_i, y_j) \in E \quad x_i \in V_1 \text{ и } y_j \in V_2 \text{ или обратно.}$
- **Свързан** - граф, на който между всеки два възела има поне един път.
- **Несвързан граф** – въвежда се понятието свързани компоненти – част от графа, която е свързан граф.



Графи. Видове

- Подграф** – ако е даден един граф $G(V, E)$, то тогава $H(V_H, E_H)$ е подграф на G . $G \supseteq H$, ако $V_H \subseteq V$ и $E_H \subseteq E$.



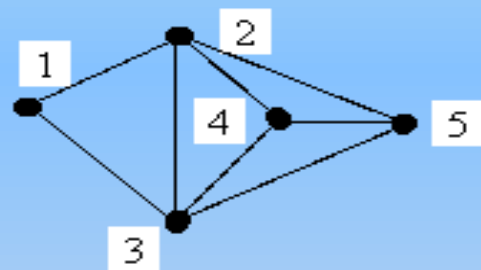
При подграф 6 не може да има подграф с повече клони – нарича се **породен** или **индуциран** – съдържа максимален брой клони.

Графи. Представяне. Външно представяне

Представяне (задаване) на графи – 2 начина:

- **Външно** – във файл. С него целта е еднозначно и максимално кратко да се представи графа за да се икономиса труд по съставянето му. Има 2 подхода:

а) Записва се номера на възела и след тире се изреждат неговите съседни;



Пр1: 1-2 3 2-3 4 5 3-1 2 4 5 4-2 3 5 5-2 3 4 # (#- край на задаването)

б) на всеки ред от файла се записват съседите на възела, чиито номер съответства на номера на реда

- Пр2: - ако възела е изолиран тогава преди # се добавя 0

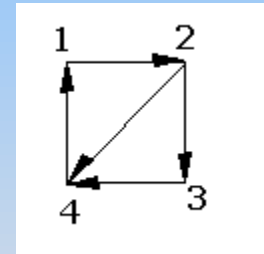
```
2 3
1 3 4 5
1 2 4 5
2 3 5
2 3 4
```

Графи. Представяне. Външно представяне

- ако графа е **тегловен** тогава задаването е:
 $1 - 2_{50}$ и 3_{75} и 4_{45} (след номера на съседния възел се записва теглото на клона)
 $3 - 4_{21}$ и $5_{80,5}$
 $4 - 5_{73} \#$
- ако графа е **насочен** правилото за възлите с по-големи номера не важи. За всеки възел се задават или само съседните клони, или тези към които има входни клони.

Пр1: - само изходящи: $1 - 2$ $2 - 3$ и 4 $4 - 1 \#$

Пр.2: - само входящи $1 - 4$ $2 - 1$ $3 - 2$ $4 - 2$ и $3 \#$



При външно задаване – необходимата памет (респ. **сложността**) е $\theta(m)$ – сложността на проверката дали два възела са съседни.

Ако се **сортират** ребрата, може да се приложи двоично търсене и сложността ще се намали на $\theta(\log_2 m)$



Графи. Представяне. Вътрешно представяне

- **Вътрешно** – Това е представяне на графа в оперативната памет по време на изпълнение на програма, ползваща структурата граф. То се оценява по следните **критерии**:
 - ✓ **колко памет** изисква дадено задаване (изразено чрез E и V);
 - ✓ операцията **дали два възела са съседни** с каква **сложност** се реализира;
 - ✓ операцията за **намирането на всички съседни възли** каква **сложност** има;

Има **два начина** за вътрешно представяне:

I^{-ви} начин – с **матрица на съседство**;

II^{-ри} начин – със **списъци на съседство**.

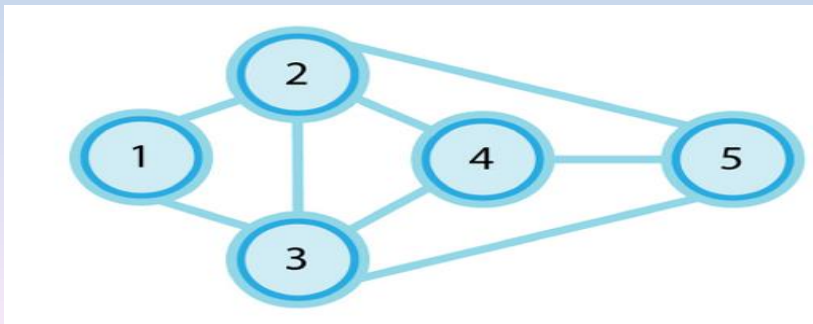


Графи. Представяне. Вътрешно представяне

- **С матрица на съседството** - за дадения граф на всеки възел съответства ред и стълб от матрицата. Тя е двоична матрица и има толкова реда и стълба, колкото са възлите:

Граф $G(V,E)$, $V=\{1,2,...,n\}$. Матрицата на съседство за $G(V,E)$ е $n \times n$ матрица A , където $a[i][j]=1$ ако има дъга от i до j и $a[i][j]=0$ в противен случай.

Създава се матрица по редовете и колоните на която се поставят номерата на възлите. При битоничните матрици в местата на връзка се записва 1, а при тегловните матрици (за тегловен граф)-теглото на реброто.

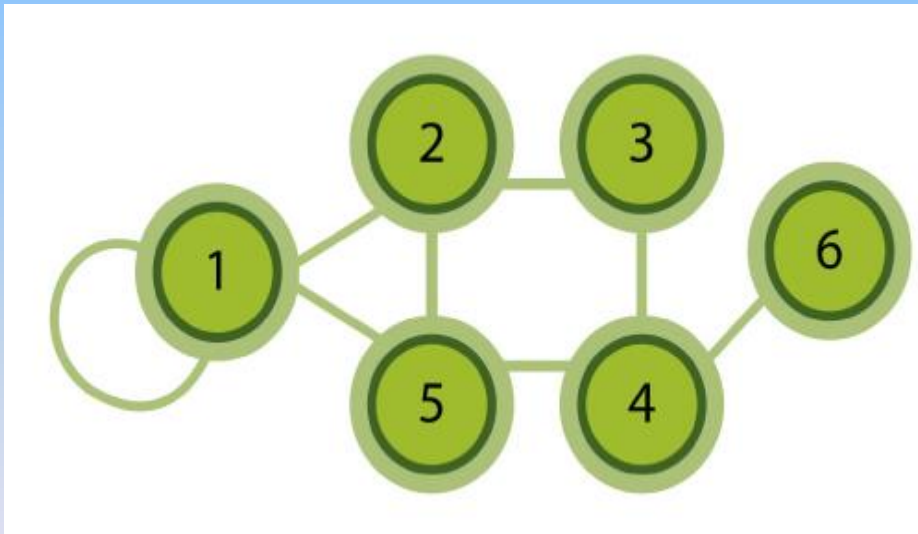


	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	1
3	1	1	0	1	1
4	0	1	1	0	1
5	0	1	1	1	0

Графи. Представяне. Вътрешно представяне

Ако в графа **няма бримки**, по главния диагонал трябва да има само **нули**. Ако графът е неориентиран, матрицата е симетрична спрямо главния диагонал.

За представянето на **ориентиран** граф са необходими n^2 клетки в паметта – доста неикономично – матрицата **не е симетрична**.



	1	2	3	4	5	6
1	1	1	0	0	1	0
2	1	0	1	0	1	0
3	0	1	0	1	0	0
4	0	0	1	1	1	0
5	1	1	0	1	0	0
6	0	0	0	1	0	0

Ако графът е тегловен, вместо единици в матрицата на съседство се записват теглата, а нулите могат да се тълкуват като 0 или ∞ .



Графи. Представяне. Вътрешно представяне

Анализ на представянето:

- **Памет** – изисква памет n^2 клетки - за много големи графи не е приложим;
- В повечето реални системи **валентността не е голяма** т.е. неикономично използване на паметта;
- По отношение на това да се намерят съседните на даден възел този метод е най-добрия;
- По отношение да се намерят всички съседни на даден възел се обхожда целия ред (стълб) и се **прави цикъл** с дължина **n**.

Графи. Представяне. Вътрешно представяне

Предимство: на представянето чрез матрица на съседство е, че се изисква **постоянно време** (само еднократен достъп до паметта) за определянето дали има или не дъга между два дадени върха. В случаите, когато с всяка дъга е свързана дължина или тегло, представянето с матрица на съседство може да се модифицира подходящо, така че $a[i][j]$ да съдържа дължината или теглото вместо единица.

Недостатък: на представянето с матрица на съседство е, че **използваната памет е n^2** клетки, даже и в случаите, когато графът има твърде малко ребра. Освен това, само еднократното преглеждане на елементите на матрицата изисква n^2 стъпки, което автоматично води до това, че използваните алгоритми ще бъдат квадратични, даже и за разреждени графи.



Графи. Представяне. Вътрешно представяне

- **Със списъци на съседство (на наследници)** - за всеки възел (връх) се задава списък на съседните му възли.

Казваме, че връх j е съседен на връх i , когато има дъга от i до j . За представянето на графа използваме масив от n елемента, в който i -тия елемент указва списъка от съседи (в произволен ред) на върха i . Следователно необходимата памет за представяне на граф с n върха и m ребра е от порядъка на $n+m$. Така се избягва недостатъка от използването на повече памет, отколкото е необходима.

Може да се реализира по **три начина**:

- ✓ Статично (с два масива)
- ✓ Полудинамично (масив+списъци)
- ✓ Динамично (списъчни структури)



Графи. Представяне. Вътрешно представяне

✓ **Статично (с два масива)** - използват се два масива: масив **“съседни”**, където последователно се изреждат номерата на съседите на всеки един възел в графа, и масив **“начало”**, където във всеки елемент се записва индекса, от който в масив **“съседни”** започва изреждането на съседите на възела, който има номер, равен на индекса на елемента в масив **“начало”**.

Масив **„съседни“** има

$2k$ елементи

(k - брой клони)

Масив **“начало”** има

$n+1$ елемента,

(n – броят възли).

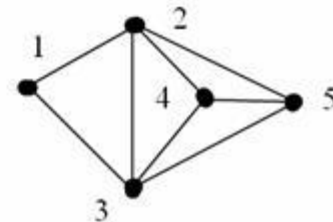
**Общата необходима
памет е:**

$n+1+2k$

“начало”		“съседни”	
i			j
1	1	2	1
2	3	3	2
3	7	1	3
4	11	2	4
5	14	4	5
6	0	5	6

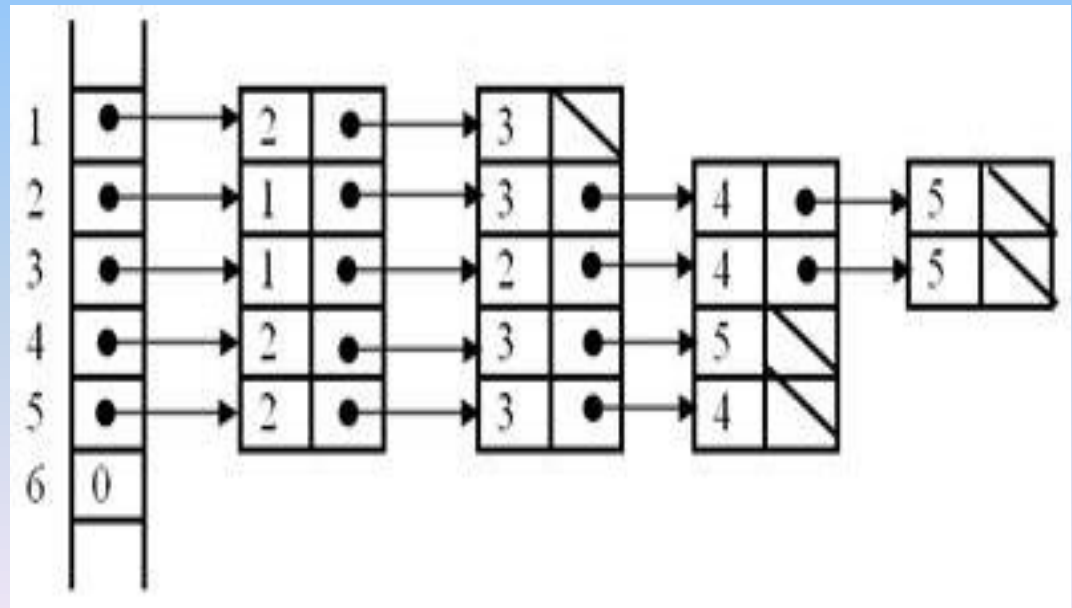
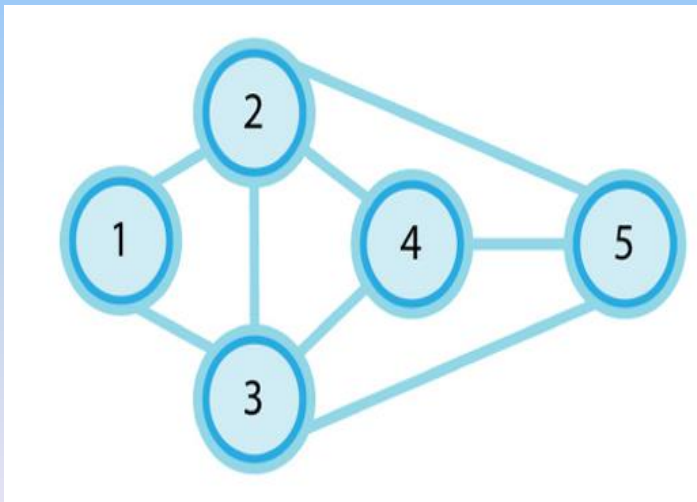
възел 1	{	2	1
възел 2	{	3	2
възел 3	{	1	3
възел 4	{	2	4
възел 5	{	4	5
		5	6

възел 3	{	1	7
		2	8
		4	9
		5	10
възел 4	{	2	11
		3	12
		5	13
възел 5	{	2	14
		3	15
		4	16
		0	17



Графи. Представяне. Вътрешно представяне

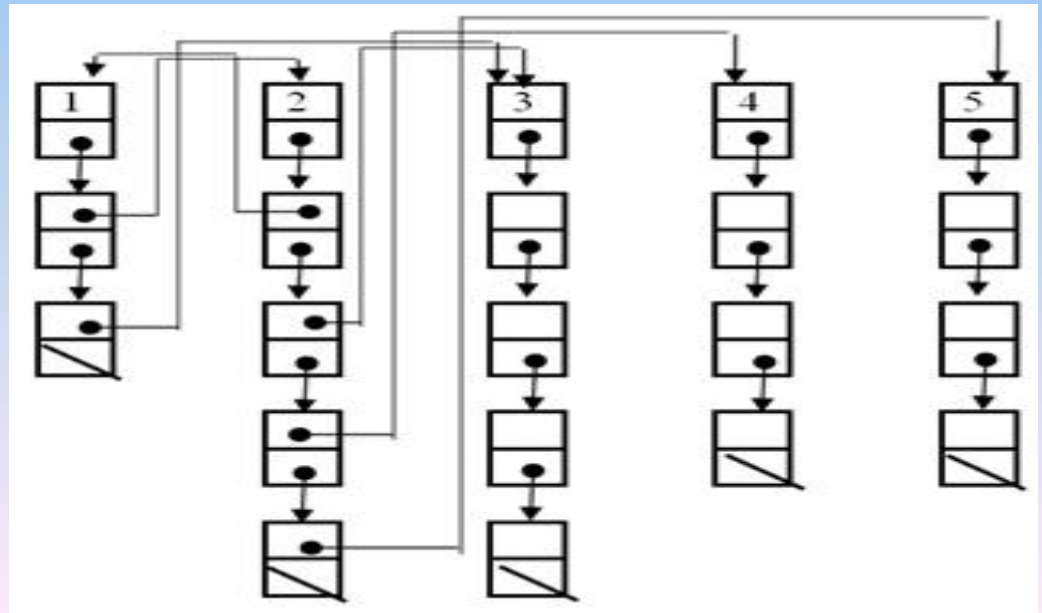
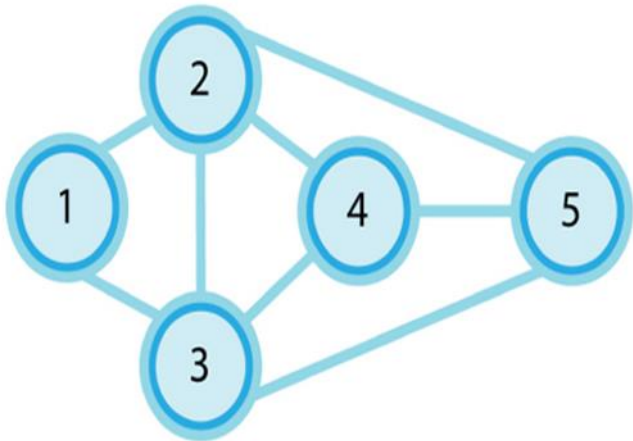
✓ **Полудинамично (с масив+списъци)** – с масив "начало", чиито елементи са указатели се задава началото на списъка на съседните възли на всеки възел. Всеки запис на съсед притежава по две полета: **номер на възела** и **указател към следващия съсед**. Указателят за следващ съсед на последния от списъка е нула.





Графи. Представяне. Вътрешно представяне

✓ **Динамично (със списъчни структури)** – за всеки възел има един запис (нар. възел) с 2 полета: едното за N : на възела, а другото (n_1) е указателно и сочи към списък от двойки указатели. Всяка двойка указатели сочи към един съседен възел и към следващата двойка. Има два вида указатели: y_1 и y_2 . Това задаване важи за свързан граф. Ако графът не е свързан, трябва да се задава началото за всяка компонента.





Графи. Построяване и прости операции

Основните операции, свързани с построяване и модифициране на граф, са следните:

- *създаване* на празен граф.
- *добавяне/премахване* на връх.
- *добавяне/премахване* на ребро.
- *проверка за съществуване* на връх
- *проверка за съществуване* на ребро
- *намиране на наследниците* на даден връх



Графи. Построяване и прости операции

Пр1: за релизация на граф чрез матрица на съседство:

```
#define MAXN 200          /* Максимален брой върхове в графа */
unsigned n;              /* Брой върхове в графа */
int A[MAXN][MAXN];       /* Матрица на теглата на графа */
A[i][i] = k;             /* Модифициране на теглото на върха i */
A[i][j] = k;             /* Добавяне на ребро с тегло k, свързващо върховете i и j */
A[j][i] = k;             /* ако графът не е ориентиран */
A[i][j] = 0;             /* Премахване на ребро, свързващо върховете i и j */
/* Проверка дали има ребро между върховете i и j */
if (A[i][j] != 0)
{
    /* има ребро */;
} else {
    /* няма ребро */;
}
/* Намиране на всички наследници на върха i */
for (k = 0; k < n; k++) if (k != i) {
    if (A[i][k] != 0) { /* върхът k е наследник на i */; }
}
```

Графи. Построяване и прости операции

Пр2: Преобразуване на представянето на граф от един вид в друг- от списък на ребра – в матрица на съседство:

/За представяне на матрицата на съседство е използван двумерен целочислен масив М. Списъкът от съседни се въвежда от клавиатурата по следния начин: от първия ред се прочитат целите числа n и m, съответно броя на върховете и броя на ребрата на графа. На всеки от останалите m реда се въвеждат двойка числа – стойностите на двата върха, свързани чрез съответното ребро. Ако графът е претеглен, за всяко ребро се въвеждат три числа, като третото е теглото на реброто./

```
int M[N][N];           //Матрицата на съседство
void Read( )           /*Функция за записване на графа в матрица на съседство*/
{ int,a,b,i;
  scanf("%d %d",&n,&m);    /*n – брой върхове, m – брой ребра*/
  for(i=0;i<m;i++)
  { scanf("%d %d",&a,&b);    //реброто (a,b)
    M[a][b]=M[b][a]=1; }    /*записване в матрицата на реброто (a,b)*/
}
```


Графи. Построяване и прости операции

Пр3: Преобразуване на представянето на граф от един вид в друг- от списък на ребра – в списък от съсед:

/За представяне на графа чрез списък на съседи се използва масив от вектори (като под вектор се има предвид стандартната структура в C++). Следващият пример показва използването на масив, описан чрез тип vector, от целочислени вектори за представяне на граф. /

```
#include <vector>                //стандартната библиотека вектор
vector <int> M[N];                //масив от вектори
void Read ()
{ int n,m,a,b,i;
  scanf("%d %d",&n,&m);          /*n – брой върхове, m – брой ребра*/
  for(i=0;i<m;i++)
  { scanf("%d %d",&a,&b);        //реброто (a,b)
    M[a].push_back(b);          /*добавяне на върха b като съсед на върха a*/
    M[b].push_back(a); }        /*добавяне на върха a като съсед на върха b*/
}
```



Графи. Алгоритми за обхождане

Има два основни алгоритъма за обхождане на графи:

- **В ширина (Breadth First Search)** - при обхождането в широчина се тръгва от някакъв начален връх и се обхождат всички негови наследници, чак след това се обхождат наследниците на наследниците.
- **В дълбочина (Depth First Search)** - при обхождането в дълбочина се тръгва от някакъв начален връх и се преминава към негов наследник, след което към наследник на наследника и така, докато се стигне връх без наследници (от който не излизат ребра). Тогава се прави връщане назад към предишния връх.



Графи. Алгоритми за обхождане – в ширина

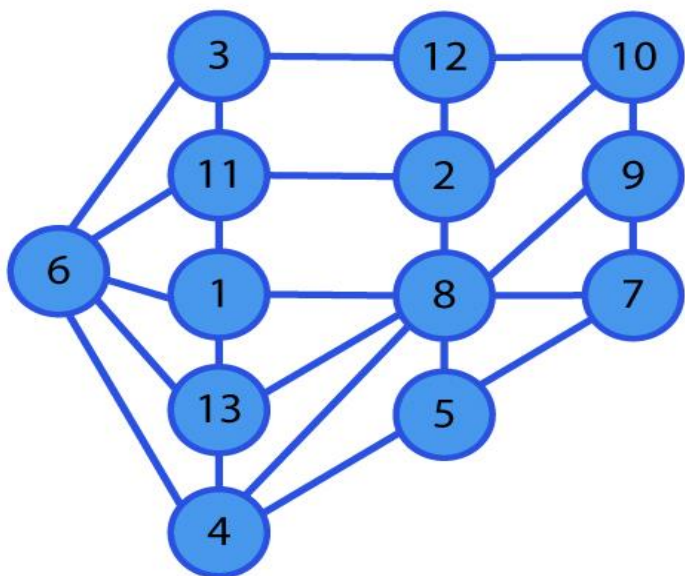
- **Обхождане в ширина (Breadth First Search)** - Характерно за този алгоритъм е, че след като се посети един връх, се обхождат всички негови непосетени съседни и чак след това се преминава към обхождане на техните съседни и т.н. За реализацията на това обхождане се използва опашка, в която се записват върховете на графа.

Алгоритъм:

1. *Разглежда се върхът i .*
2. *Добавя се върхът i в опашката от върхове.*
3. *Докато опашката не е празна се повтаря следното:*
 3. 1. *Изтрива се първият елемент от опашката.*
 3. 2. *Разглеждат се всички негови непосетени съседни и се добавят в опашката.*

Графи. Алгоритми за обхождане – в ширина

- Графът е представен чрез **матрица на съседство**. Въвеждат се двойки свързани върхове, като за **край на въвеждане** служат **две нули**. Обхождането става чрез функцията **bfs**. Като **помощен** масив се използва масива **used**, който служи за маркиране на посетените върхове. Тук е необходимо да се **използва опашка от цели числа**, като за реализацията ѝ е използван типа **queue** от стандартната библиотека на езика C++.



Възел 6 е с ниво 0. Неговите съседни са с ниво 1:
3, 11, 1, 13 и 4.

Непосетените съседни на тези възли са от ниво 2:
12 (съсед на 3)
2 (съсед на 11)
8 (съсед на 1)
5 (съсед на 4)

Непосетените съседни на тези възли са от ниво 3:
10 (съсед на 12)
9 и 7 (съседни на 8)



Графи. Алгоритми за обхождане – в ширина

Реализация на език C++:

```
#include <iostream>
#include <cstdio>
#include <queue>
#include <vector>
using namespace std;
int n,v,used[101];
queue<int>q;
vector<int>a[101];
void bfs()
{
    q.push(v);
    used[v]=1;
    while(!q.empty())
    {
        int t=q.front();
        q.pop();
        printf("%d ",t);
```

// докато опашката не е празна





Графи. Алгоритми за обхождане – в ширина

Реализация на език C++: /продължение/

```
for(int i=0; i<a[t].size(); i++)
{
    if(used[a[t][i]]==0)                /* ако не е обходен */
    {
        q.push(a[t][i]);
        used[a[t][i]]=1;
    } } }
int main()
{
    scanf("%d %d",&n,&v);                /* дължина на графа и връх, от който да обхождаме */
    int b,c;
    while(b!=0 || c!=0)
    {    scanf("%d %d",&b,&c);
        a[b].push_back(c);
        a[c].push_back(b);    }
    bfs();
    printf("\n");
    return 0;
}
```

Графи. Алгоритми за обхождане – в дълбочина

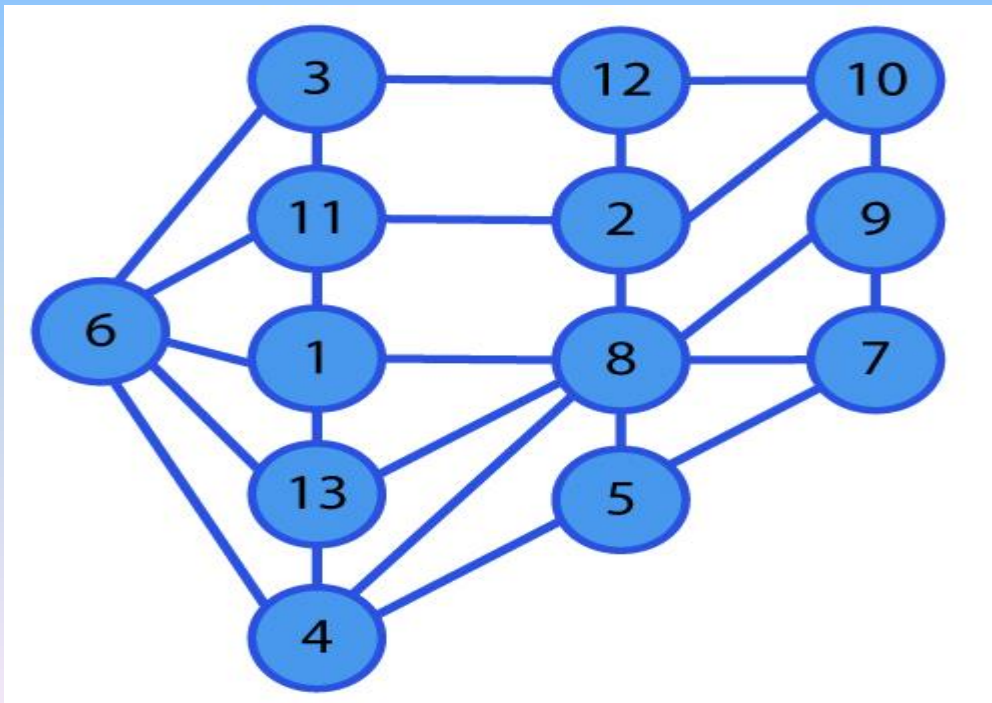
- **Обхождане в дълбочина (Depth First Search)**- Алгоритъмът за обхождане в дълбочина наред с това, че е неразделна част от някои по-сложни алгоритми, е основна идея при един от фундаменталните методи за решаване на изчерпващи задачи – търсене с връщане (backtracking).

Наименованието на алгоритъма идва от това, че при посещение на връх се стремим да се “спуснем” от него колкото може по-надолу. За реализацията му е най-удобно да се използва рекурсивна функция, която може да се опише със следният **Алгоритъм**:

1. *Разглежда се връх i .*
2. *Маркира се като посетен.*
3. *За всеки непосетен връх j се изпълнява стъпка 1.*
4. *Алгоритъмът завършва след като всички върхове на графа са посетени.*

Графи. Алгоритми за обхождане – в дълбочина

Тук графът отново е представен чрез **матрица на съседство**. Въвеждат се двойки свързани върхове, като за **край на въвеждане** служат **две нули**. Обхождането става чрез **рекурсивната функция dfs**. Като **помощен масив** се използва масива **used**, който служи за маркиране на посетените върхове.





Графи. Алгоритми за обхождане – в дълбочина

Реализация на език C++:

```
#include <iostream>
#include <cstdio>
#include <vector>
#include <algorithm>
using namespace std;
int n,v,used[101];
vector<int>a[101];
void dfs(int k)
{
    printf("%d ",k);
    for(int i=0; i<a[k].size(); i++)
    {
        if(used[a[k][i]]) continue;
        used[a[k][i]]=1;
        dfs(a[k][i]);
    }
}
```





Графи. Алгоритми за обхождане – в ширина

Реализация на език C++: /продължение/

```
int main()
{
    scanf("%d %d",&n,&v); /* Брой върхове на графа и връх, от който да обхождаме */
    int b,c;
    while(b!=0 || c!=0)
    {
        scanf("%d %d",&b,&c);
        a[b].push_back(c);
        a[c].push_back(b);
    }
    for (int i=0; i<n; i++) sort(a[i].begin(),a[i].end());
    used[v]=1;
    dfs(v);
    return 0;
}
```



Графи. Други алгоритми и задачи

Интересни задачи и алгоритми свързани с графи са:

- алгоритъмът на **Дейкстра** за **намиране на най-къс път** между два върха;
- намиране на **минимално скелетно дърво**;
- откриването на **Хамилтонов цикъл** в граф;
- конструирането на **Ойлеров път** в граф (За първи път този проблем е разглеждан от Леонард Ойлер, докато решава проблема за Кьонигсбергските мостове);
- и други алгоритми за графи.

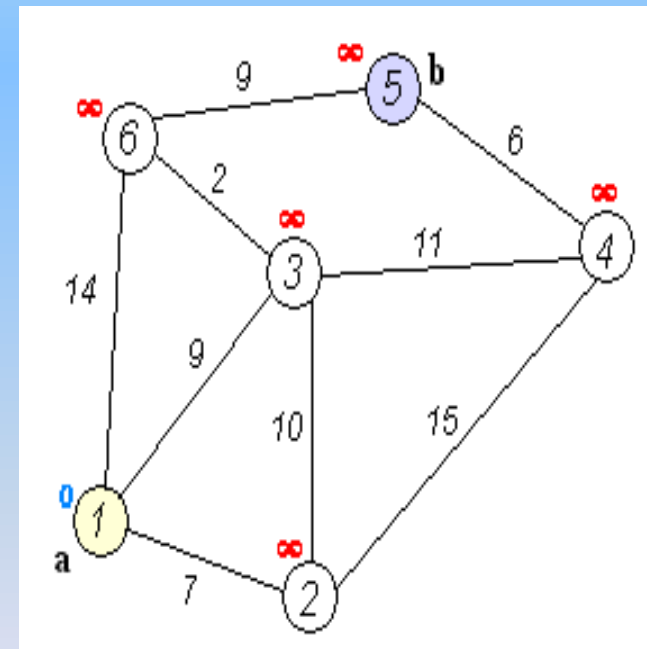


Графи. Други алгоритми и задачи.

Алгоритъм на Дейкстра

Алгоритъмът на Дейкстра за намиране на най-къс път между два върха:

Нека е даден претеглен ориентиран граф $G(V, E)$ с n върха. За да бъде приложен алгоритъмът, теглата на ребрата $f(i, j)$ трябва да бъдат **положителни** числа. Търси се минималният път от фиксиран връх $s \in V$ до всички останали върхове на графа. С $\varphi(s, i)$ ще означим **дължината на минималния път** от s до i . Най-общо, алгоритъмът на Дейкстра се основава на следния принцип: За да намерим $\varphi(s, i)$, трябва да намерим минималното измежду $\varphi(s, j) + f(j, i)$, за всяко $j, j \neq i$. Така, на всеки връх i от графа се присвоява временна стойност $d[i]$, която представлява горна граница за $\varphi(s, i)$. В процеса на работа на алгоритъма тази стойност намалява, докато накрая $d[i]$ стане точно равна на $\varphi(s, i)$:





Графи. Други алгоритми и задачи.

Алгоритъм на Дейкстра

- алгоритъмът на **Дейкстра** за **намиране на най-къс път** между два върха:

Алгоритъм на Дейкстра :

1) Инициализираме масив $d[]$ по следния начин:

$d[i] = A[s][i]$ за всеки съсед $i \in V$ на s .

$d[i] = MAX_VALUE$, за всеки връх i , несъседен на s .

След приключване на алгоритъма $d[i] == MAX_VALUE$ т.к. между s и i няма път.

2) Въвеждаме множество T , което в началото съдържа всички върхове на графа, без s : $T = V \setminus \{s\}$

3) Докато T съдържа поне един връх i , за който $d[i] < MAX_VALUE$:

3.1) Избираме връх $j \in T$ такъв, че $d[j]$ да бъде минимално.

3.2) Изключваме j от T : $T = T \setminus \{j\}$

3.3) За всяко $i \in T$ изпълняваме $d[i] = \min(d[i], d[j] + A[j][i]);$



Графи. Други алгоритми и задачи.

Алгоритъм на Дейкстра

```
function Dijkstra(Graph, source):  
    for each vertex v in Graph:  
        dist[v] := infinity ;  
  
        previous[v] := undefined ;  
    end for  
  
    dist[source] := 0 ;  
    Q := the set of all nodes in Graph ;  
  
    while Q is not empty:  
        u := vertex in Q with smallest distance in dist[] ;  
        remove u from Q ;  
        if dist[u] = infinity:  
            break ;  
        end if  
  
        for each neighbor v of u in Q:  
  
            alt := dist[u] + dist_between(u, v) ;  
            if alt < dist[v]:  
                dist[v] := alt ;  
                previous[v] := u ;  
                decrease-key v in Q;  
            end if  
        end for  
    end while  
return dist;
```

// Initializations
// Unknown distance function from
// source to v
// Previous node in optimal path
// from source

// Distance from source to source
// All nodes in the graph are put in a
// Priority Queue Q
// The main loop
// Start node in first case

// all remaining vertices are
// inaccessible from source

// where v has not yet been
// removed from Q.

// Relax (u,v,a)

// Reorder v in the Queue



Графи. Други алгоритми и задачи.

Алгоритъм на Дейкстра

Сложността на алгоритъма е $\theta(n^2)$. При по-внимателно подбиране на структурите от данни (например, ако използваме пирамида ([Cormen, Leiserson, Rivest-1997])) сложността може да се понижи до $\theta(n \cdot \log_2 n)$.

***Забележка:** Алгоритъмът на Дейкстра няма да работи правилно, ако в графа има ребра с отрицателни тегла.*



Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

- алгоритъмът на **Крускал** за **намиране на минимално скелетно (покриващо) дърво**:

Да си представим група острови, които искаме да свържем с мостове по такъв начин, че да може да се стигне от всеки остров до всеки друг в групата. Тъй като строенето на мостовете е свързано с определени разходи, целта, която ще си поставим, ще бъде множеството от мостове, което строим, да има минимална цена. Ще приемем, че за всяка двойка острови знаем дали може да се свърже директно с мост и колко струва той.

Изказана в термините от теорията на графите, горната задача ще изглежда така: Даден е неориентиран граф $G(V, E)$, в който върховете от множеството V са островите, а ребро (i, j) съществува т.к е възможно да се построи мост между островите i и j . Теглото $f(i, j)$ определя евентуалната му цена. Търси се покриващо (обхващащо) дърво $T(V, D)$ на G с минимална сума от теглата на участващите в D ребра.



Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

Алгоритъм на Крускал за намиране на МПД:

- 1) Създаваме n множества, като в i -тото множество поставяме i -тия връх от графа.
- 2) Сортираме ребрата на графа във възходящ ред (по теглата им).
- 3) Създаваме празно дърво $T(V,0)$. След приключване на алгоритъма T ще бъде търсеното покриващо дърво.
- 4) Последователно ($n-1$ пъти) добавяме ребро $(i,j) \in E$ към T , така че да бъде изпълнено:
 - а) теглото $f(i, j)$ да бъде възможно най-малко (разполагаме със сортиран по теглата списък на ребрата от графа)
 - б) върховете i и j да се намират в различни множестваСлед всяко добавено ребро (i,j) обединяваме множествата, в които се намират i и j .



Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

```
#include <stdio.h>

#define MAXN 200                                /* Максимален брой върхове в графа */
#define NO_PARENT (unsigned)(-1)
#define MAXM 2000                                /* Максимален брой ребра в графа */
const unsigned n = 9;                          /* Брой върхове в графа */
const unsigned m = 14;                          /* Брой ребра в графа */

struct arc {
    unsigned i, j;
    int f;
};

struct arc S[MAXM] = {                          /* Списък от ребрата на графа и техните тегла */
    { 1, 2, 1},
    { 1, 4, 2},
    { 2, 3, 3},
    { 2, 5, 13},
    { 3, 4, 4},
```





Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

```
{ 3, 6, 3},  
{ 4, 6, 16},  
{ 4, 7, 14},  
{ 5, 6, 12},  
{ 5, 8, 1},  
{ 5, 9, 13},  
{ 6, 7, 1},  
{ 6, 9, 1}
```

```
};
```

```
int prev[MAXN + 1];
```

```
int getRoot(int i)
```

```
{ int root, savei;
```

```
root = i;
```

/ намиране на корена на дървото */*

```
while (NO_PARENT != prev[root]) root = prev[root];
```

```
while (i != root) {
```

/ свиване на пътя */*

```
savei = i;
```





Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

```
i = prev[i];  
prev[savei] = root; }  
return root; }  
  
void kruskal(void)  
{ int MST = 0;  
  unsigned i, j;  
  sort(S, 0, m);      /* сортира списъка с ребрата в нарастващ ред по тяхното тегло */  
  printf("Ребрата, които участват в минималното покриващо дърво:\n");  
  for (i = 0; i < m; i++) {  
    int r1 = getRoot(S[i].i);  
    int r2 = getRoot(S[i].j);  
    if (r1 != r2) {  
      printf("(%u,%u) ", S[i].i, S[i].j);  
      MST += S[i].f;  
      prev[r2] = r1; }  
  }
```





Графи. Други алгоритми и задачи.

Алгоритъм на Крускал

```
printf("\nЦената на минималното покриващо дърво е %d.\n", MST);  
}  
  
int main() {  
    unsigned i;  
    for (i = 0; i < n + 1; i++) prev[i] = NO_PARENT;  
    kruskal();  
    return 0;  
}
```

Брой елементарни операции на

търсене и обединение

n^2

$n \cdot \log n$

$n \cdot \log^* n$

Балансиране на дървото

НЕ

ДА

ДА

Свиване на пътя

НЕ

НЕ

ДА

Общата сложност на алгоритъма е: $\theta(m \cdot \log^2 m + n \cdot \log^2 n)$.